Experiment 7

Establish a local Blockchain using Ganache and deploy a Smart Contract through MetaMask and Remix IDE

Aim

To start a local Ethereum blockchain with Ganache, connect its accounts to MetaMask and Remix IDE, deploy a set of Solidity smart contracts modeling a 4-stage supply chain (Farmer → Distributor → Retailer → Customer) through MetaMask, interact with them, and check the transactions on the Ganache chain.

Tasks Performed

1. Installed and started Ganache (CLI v7.9.2) as a local blockchain with ten funded accounts.
2. Connected the Ganache accounts to MetaMask: added the Ganache network and imported four accounts as Farmer, Distributor, Retailer and Customer.
3. Connected Remix IDE to MetaMask (Environment: Browser Extension).
4. Wrote the mini-project contracts: a four-stage supply chain (`FarmerContract`, `DistributorContract`, `RetailerContract`, `CustomerContract`).
5. Compiled the contracts and deployed them through the MetaMask accounts on Ganache.
6. Checked the transaction details on the Ganache chain (its log and a transaction list read over RPC).
7. Interacted with the contracts: moved one batch through all four stages, and tried a repeat purchase that reverted.

Theory

Ganache is a personal Ethereum blockchain for development and testing. It mines every transaction instantly, comes with pre-funded test accounts (1000 ETH each) and a deterministic mnemonic, and lets a developer inspect blocks, transactions and balances. The original Ganache desktop UI is no longer maintained, so this experiment uses the same engine through its command-line version (`ganache`, v7.9.2), which listens on `http://127.0.0.1:7545` with chain ID 5777 and prints every RPC call and mined transaction in its terminal.

MetaMask is a browser wallet that holds private keys and signs transactions. A local chain is added to it as a custom network (name, RPC URL, chain ID, currency symbol), and Ganache accounts are imported into it by private key. **Remix IDE** talks to MetaMask through the *Browser Extension* (Injected Provider) environment: Remix builds the transaction, MetaMask shows an approval screen, and only after

Confirm

is the transaction signed and sent to the chain.

Steps followed:

1. Start Ganache and note its accounts and private keys.
2. Add Ganache as a custom network in MetaMask and import four Ganache accounts, renamed Farmer, Distributor, Retailer and Customer.
3. Connect Remix IDE to MetaMask (Environment: Browser Extension → MetaMask) and give it those four accounts.
4. Compile the four contracts and deploy them through MetaMask.
5. Call the contract functions, switching MetaMask to the right role's account for each call, and approve each request.
6. Check the transaction details on the Ganache chain.

The task's contract is a 4-stage supply-chain traceability system, four contracts chained by constructor-injected addresses:

- **`FarmerContract`** — `addBatch(id, cropName, quantity)` records a new batch on-chain with the farmer's address and a timestamp; `getBatch` reads it back.
- **`DistributorContract`** — holds a reference to `FarmerContract`; `receiveBatch(id)` calls into it through the `IFarmerContract` interface to pull the batch's data and record that this distributor received it.
- **`RetailerContract`** — the same pattern one stage further down, reading from `DistributorContract` through `IDistributorContract`.
- **`CustomerContract`** — the final stage, reading from `RetailerContract` through `IRetailerContract` and recording the purchase.

Each stage's `require` guards (the batch must exist upstream and must not already have been received or purchased at this stage) let a batch move forward through the chain only once, and each contract trusts only the *interface*

of the contract before it.

Tools used: Ganache CLI 7.9.2 (`--server.port 7545 --chain.chainId 5777 --wallet.deterministic`), MetaMask 13.53, Remix IDE (web, v2.6.8) with Solidity compiler 0.8.34 and the EVM version set to Shanghai, which is the newest hard fork Ganache supports. The terminal screenshots were taken from a real terminal running in the browser.

Program

The four contracts, in the order they are deployed (each takes the previous contract's address as a constructor argument):

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

contract FarmerContract {
    struct Batch {
        uint256 batchID;
        string cropName;
        uint256 quantity;
        uint256 timestamp;
        address farmer;
    }
    mapping(uint256 => Batch) public batches;
    event BatchCreated(
        uint256 batchID,
        string cropName,
        uint256 quantity,
        uint256 timestamp,
        address farmer
    );
    function addBatch(uint256 _batchID, string memory
_cropName, uint256 _quantity) public {
        require(_batchID != 0, "Invalid batch ID");
        require(batches[_batchID].batchID == 0, "Batch already
exists!");

        batches[_batchID] = Batch({
            batchID: _batchID,
            cropName: _cropName,
            quantity: _quantity,
            timestamp: block.timestamp,
            farmer: msg.sender
        });
        emit BatchCreated(_batchID, _cropName, _quantity,
block.timestamp, msg.sender);
    }
    function getBatch(uint256 _batchID) public view returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address
    ) {
        Batch memory b = batches[_batchID];
        require(b.batchID != 0, "Batch does not exist!");
        return (b.batchID, b.cropName, b.quantity, b.timestamp,
b.farmer);
    }
}

// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

interface IFarmerContract {
    function getBatch(uint256 _batchID) external view returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address
    );
}

contract DistributorContract {
    struct ReceivedBatch {
        uint256 batchID;
        string cropName;
        uint256 quantity;
        uint256 timestamp;
        address farmer;
        address distributor;
    }
    mapping(uint256 => ReceivedBatch) public receivedBatches;
    IFarmerContract public farmerContract;
    event BatchReceived(
        uint256 batchID,
        string cropName,
        uint256 quantity,
        address farmer,
        address distributor,
        uint256 timestamp
    );
    constructor(address _farmerContractAddress) {
        farmerContract =
IFarmerContract(_farmerContractAddress);
    }
    function receiveBatch(uint256 _batchID) public {
        (
```

```
            uint256 id,
            string memory cropName,
            uint256 quantity,
            , // ignore farmer timestamp
            address farmer
        ) = farmerContract.getBatch(_batchID);
        require(id != 0, "Batch does not exist in
FarmerContract");
        require(receivedBatches[_batchID].batchID == 0, "Batch
already received!");
        receivedBatches[_batchID] = ReceivedBatch({
            batchID: id,
            cropName: cropName,
            quantity: quantity,
            timestamp: block.timestamp,
            farmer: farmer,
            distributor: msg.sender
        });
        emit BatchReceived(id, cropName, quantity, farmer,
msg.sender, block.timestamp);
    }
    function getReceivedBatch(uint256 _batchID) public view
returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address
    ) {
        ReceivedBatch memory rb = receivedBatches[_batchID];
        require(rb.batchID != 0, "Batch not received!");
        return (
            rb.batchID,
            rb.cropName,
            rb.quantity,
            rb.timestamp,
            rb.farmer,
            rb.distributor
        );
    }
}

// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

interface IDistributorContract {
    function getReceivedBatch(uint256 _batchID) external view
returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address
    );
}

contract RetailerContract {
    struct RetailBatch {
        uint256 batchID;
        string cropName;
        uint256 quantity;
        uint256 timestamp;
        address farmer;
        address distributor;
        address retailer;
    }
    mapping(uint256 => RetailBatch) public retailBatches;
    IDistributorContract public distributorContract;
    event BatchReceivedAtRetailer(
        uint256 batchID,
        string cropName,
        uint256 quantity,
        address farmer,
        address distributor,
        address retailer,
        uint256 timestamp
    );
    constructor(address _distributorContractAddress) {
        distributorContract =
IDistributorContract(_distributorContractAddress);
    }

    function receiveFromDistributor(uint256 _batchID) public {
        (
            uint256 id,
            string memory cropName,
            uint256 quantity,
            , // ignore distributor timestamp
            address farmer,
            address distributor
        ) = distributorContract.getReceivedBatch(_batchID);
        require(id != 0, "Batch does not exist in
DistributorContract");
        require(retailBatches[_batchID].batchID == 0, "Batch
already received at Retailer!");
        retailBatches[_batchID] = RetailBatch({
            batchID: id,
            cropName: cropName,
            quantity: quantity,
            timestamp: block.timestamp,
            farmer: farmer,
            distributor: distributor,
            retailer: msg.sender
        });
        emit BatchReceivedAtRetailer(
            id,
            cropName,
            quantity,
            farmer,
            distributor,
            msg.sender,
            block.timestamp
        );
    }
    function getRetailBatch(uint256 _batchID) public view
returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address,
        address
    ) {
        RetailBatch memory rb = retailBatches[_batchID];
        require(rb.batchID != 0, "Batch not received at
retailer!");

        return (
            rb.batchID,
            rb.cropName,
            rb.quantity,
            rb.timestamp,
            rb.farmer,
            rb.distributor,
            rb.retailer
        );
    }
}

// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

interface IRetailerContract {
    function getRetailBatch(uint256 _batchID) external view
returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address,
        address
    );
}

contract CustomerContract {
    struct PurchasedBatch {
        uint256 batchID;
        string cropName;
        uint256 quantity;
        uint256 timestamp;
        address farmer;
        address distributor;
        address retailer;
        address customer;
    }
    mapping(uint256 => PurchasedBatch) public purchasedBatches;
    IRetailerContract public retailerContract;
    event BatchPurchased(
```

```
        uint256 batchID,
        string cropName,
        uint256 quantity,
        address farmer,
        address distributor,
        address retailer,
        address customer,
        uint256 timestamp
    );
    constructor(address _retailerContractAddress) {
        retailerContract =
IRetailerContract(_retailerContractAddress);
    }
    function purchaseFromRetailer(uint256 _batchID) public {
        (
            uint256 id,
            string memory cropName,
            uint256 quantity,
            , // ignore retailer timestamp
            address farmer,
            address distributor,
            address retailer
        ) = retailerContract.getRetailBatch(_batchID);
        require(id != 0, "Batch does not exist in
RetailerContract");
        require(purchasedBatches[_batchID].batchID == 0, "Batch
already purchased!");
        purchasedBatches[_batchID] = PurchasedBatch({
            batchID: id,
            cropName: cropName,
            quantity: quantity,
            timestamp: block.timestamp,
            farmer: farmer,
            distributor: distributor,
            retailer: retailer,
            customer: msg.sender
        });
        emit BatchPurchased(
            id,
            cropName,
            quantity,
            farmer,
            distributor,
            retailer,
            msg.sender,
            block.timestamp
        );
    }
    function getPurchasedBatch(uint256 _batchID) public view
returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address,
        address,
        address
    ) {
        PurchasedBatch memory pb = purchasedBatches[_batchID];
        require(pb.batchID != 0, "Batch not purchased!");
        return (
            pb.batchID,
            pb.cropName,
            pb.quantity,
            pb.timestamp,
            pb.farmer,
            pb.distributor,
            pb.retailer,
            pb.customer
        );
    }
}
```

Output

```
ganache v7.9.2 (@ganache/cli: 0.10.2, @ganache/core: 0.10.2)
Starting RPC server

Available Accounts
==================
(0) 0x90F8bf6A479f320ead074411a4B0e7944Ea8c9C1 (1000 ETH)
(1) 0xFFcf8FDEE72ac11b5c542428B35EEF5769C409f0 (1000 ETH)
(2) 0x22d491Bde2303f2f43325b2108D26f1eAbA1e32b (1000 ETH)
(3) 0xE11BA2b4D45Eaed5996Cd0823791E0C93114882d (1000 ETH)
(4) 0xd03ea8624C8C5987235048901fB614fDcA89b117 (1000 ETH)
(5) 0x95cED938F7991cd0dFcb48F0a06a40FA1aF46EBC (1000 ETH)
(6) 0x3E5e9111Ae8eB78Fe1CC3bb8915d5D461F3Ef9A9 (1000 ETH)
(7) 0x28a8746e75304c0780E011BEd21C72cD78cd535E (1000 ETH)
(8) 0xACa94ef8bD5ffEE41947b4585a84BdA5a3d3DA6E (1000 ETH)
(9) 0x1dF62f291b2E969fB0849d99D9Ce41e2F137006e (1000 ETH)

Private Keys
==================
(0) 0x4f3edf983ac636a65a842ce7c78d9aa706d3b113bce9c46f30d7d21715b23b1d
(1) 0x6cbed15c793ce57650b9877cf6fa156fbef513c4e6134f022a85b1ffdd59b2a1
(2) 0x6370fd033278c143179d81c5526140625662b8daa446c22ee2d73db3707e620c
(3) 0x646f1ce2fdad0e6deeeb5c7e8e5543bdde65e86029e2fd9fc169899c440a7913
(4) 0xadd53f9a7e588d003326d1cbf9e4a43c061aadd9bc938c843a79e7b4fd2ad743
(5) 0x395df67f0c2d2d9fe1ad08d1bc8b6627011959b79c53d7dd6a3536a33ab8a4fd
(6) 0xe485d098507f54e7733a205420dfddbe58db035fa577fc294ebd14db90767a52
(7) 0xa453611d9419d0e56f499079478fd72c37b251a94bfde4d19872c44cf65386e3
(8) 0x829e924fdf021ba3dbbc4225edfece9aca04b929d6e75613329ca6f1d31c0bb4
(9) 0xb0057716d5917badaf911b193b12b910811c1497b5bada8d7711f758981c3773

HD Wallet
==================
Mnemonic:      myth like bonus scare over problem client lizard pioneer submit female collect
Base HD Path:  m/44'/60'/0'/0/{account_index}

Default Gas Price
==================
2000000000

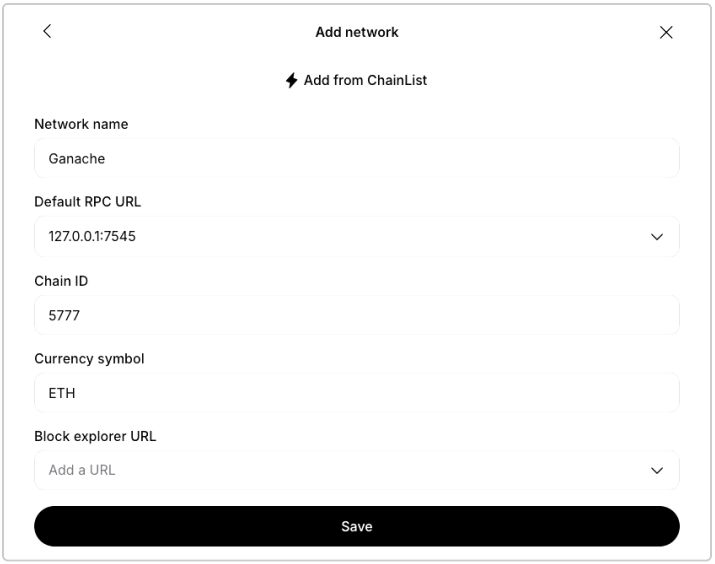
BlockGas Limit
==================
30000000

Call Gas Limit
==================
50000000

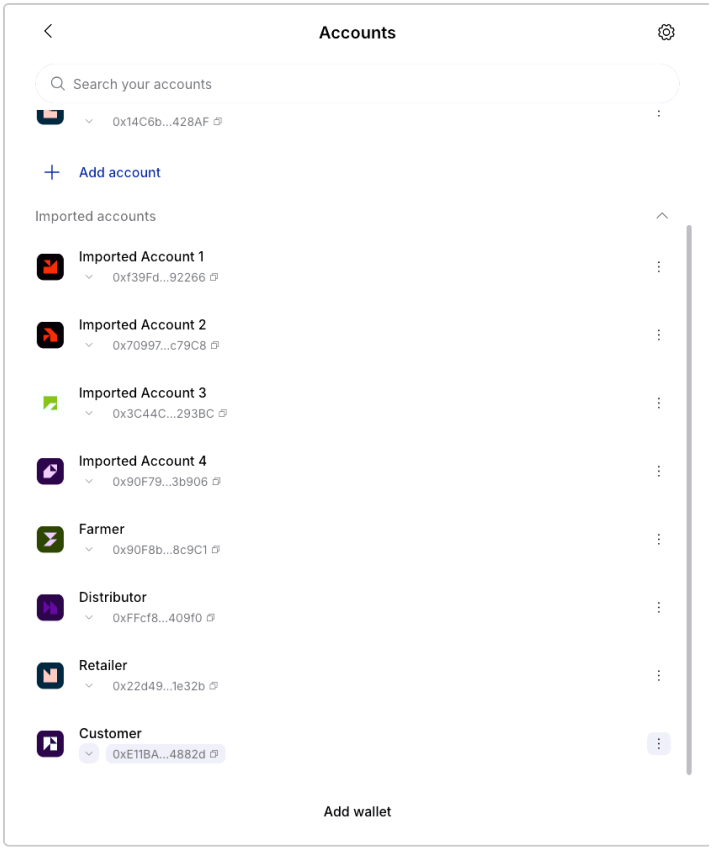
Chain
==================
Hardfork: shanghai
Id:       5777

RPC Listening on 127.0.0.1:7545
```

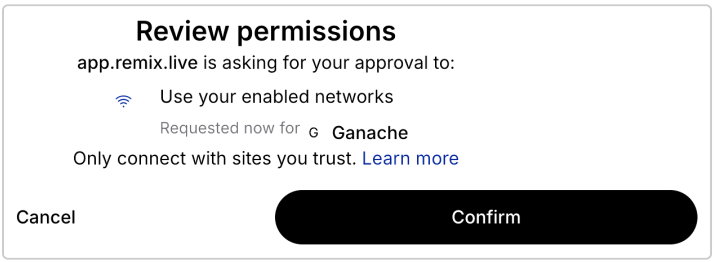
Task 1: `ganache` starts a local chain with ten accounts holding 1000 ETH each. It reports “RPC Listening on 127.0.0.1:7545” with chain ID 5777.



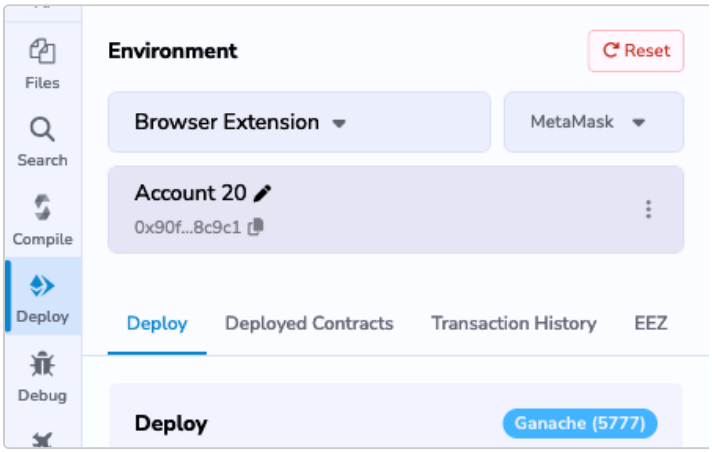
Task 2: Ganache added to MetaMask as a custom network (RPC `127.0.0.1:7545`, chain ID 5777, symbol ETH).



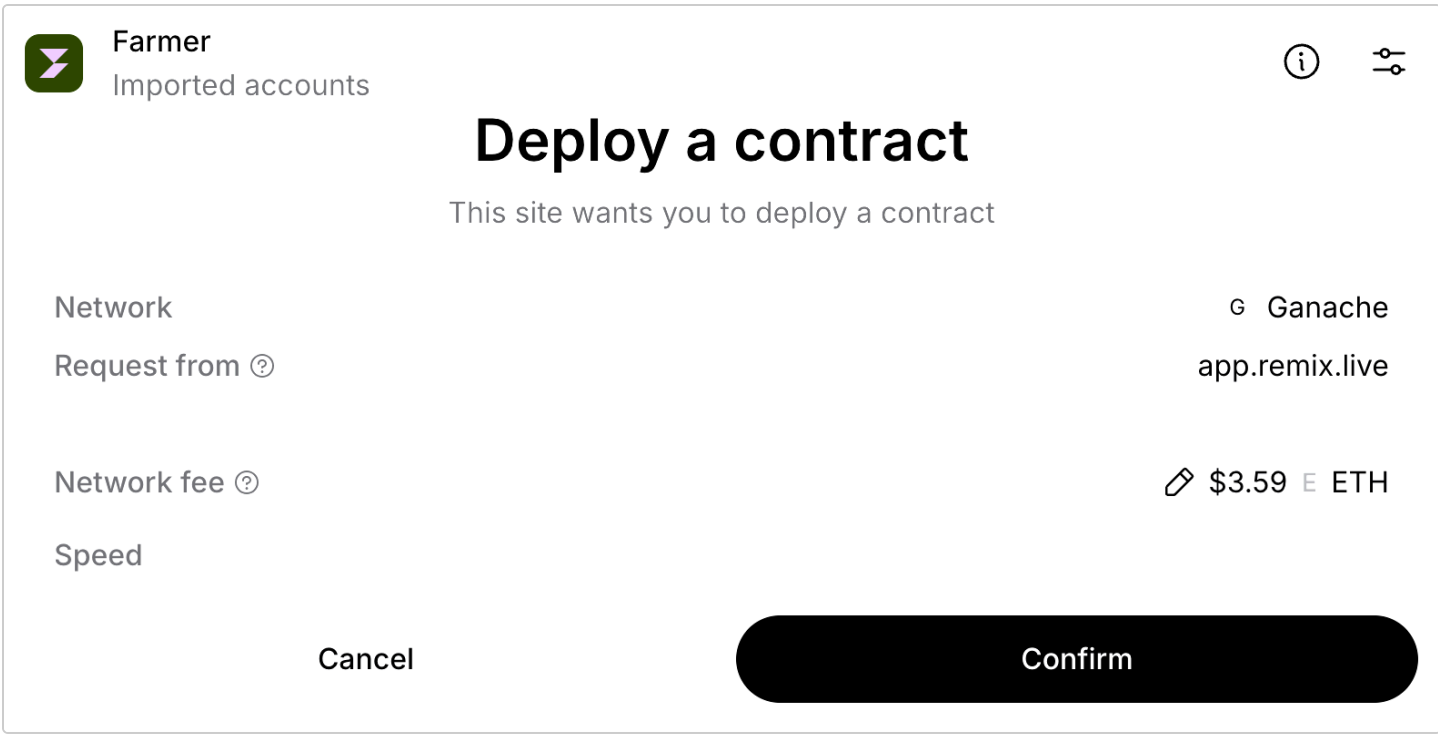
Task 2: four Ganache accounts imported into MetaMask and renamed Farmer, Distributor, Retailer and Customer.



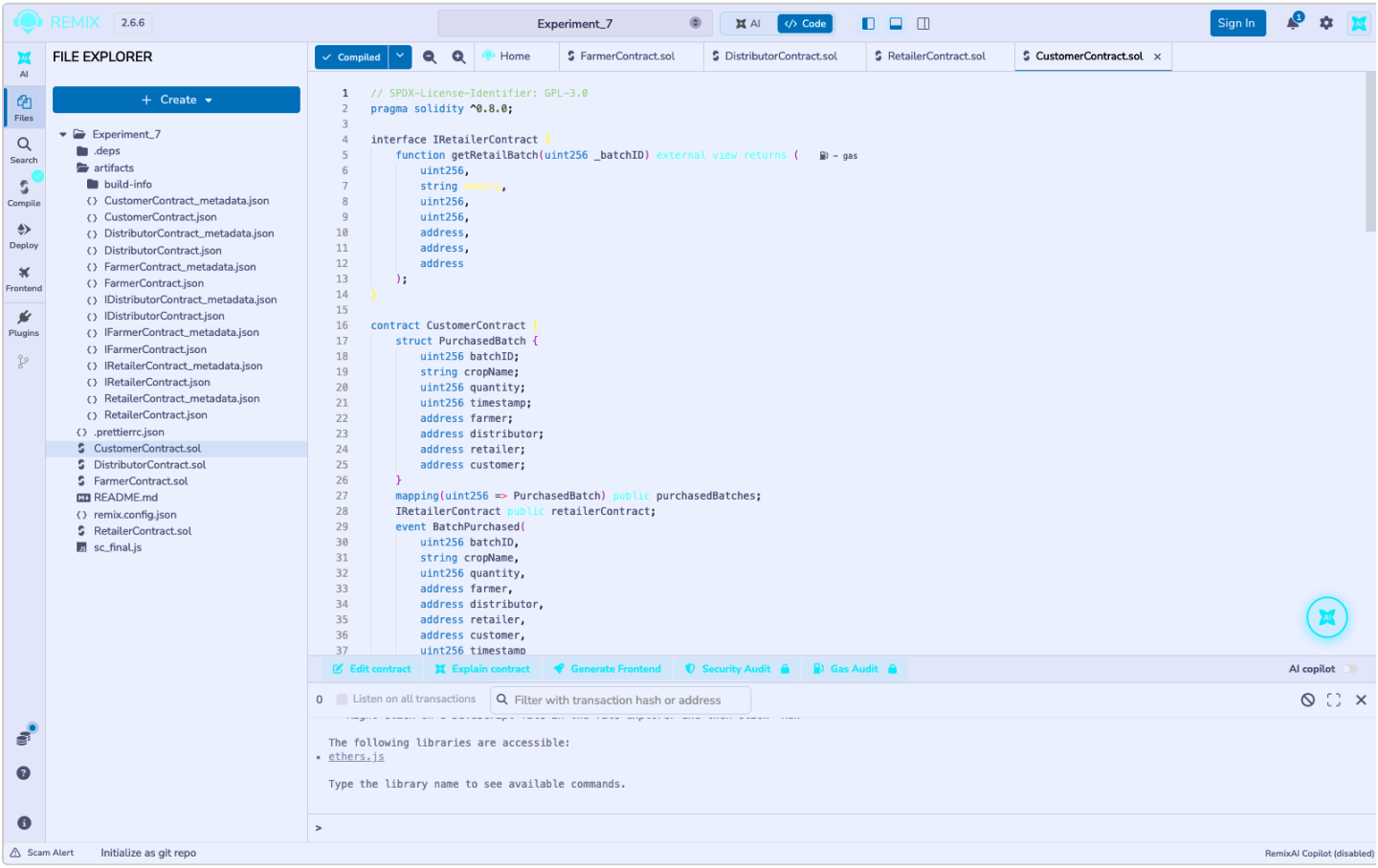
Task 3: MetaMask asks to approve app.remix.live's use of the Ganache network.



Task 3: Remix shows Environment "Browser Extension / MetaMask" and the network label "Ganache (5777)".

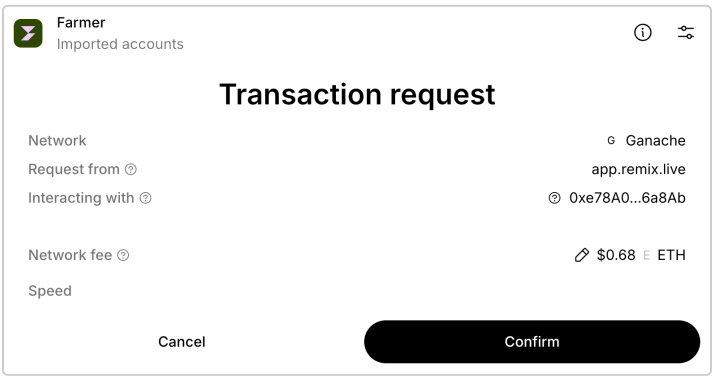


Task 5: MetaMask shows "Deploy a contract" for `FarmerContract` from app.remix.live on the Ganache network. The other three contracts were deployed the same way, each with the previous contract's address as its constructor argument.

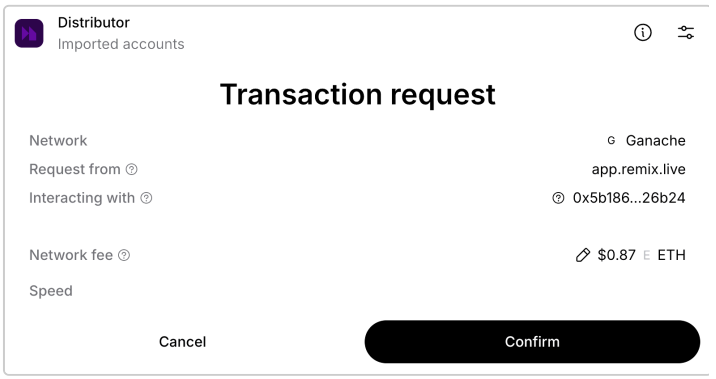


Task 5: all four contracts compile with no errors; the green tick on the Solidity Compiler icon confirms it.

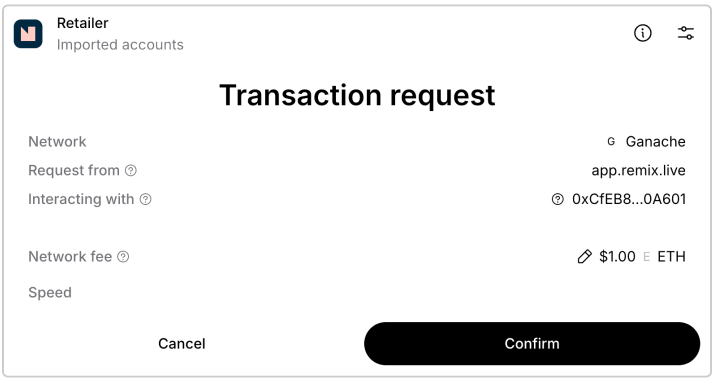
Moving one batch through the four stages (Task 7)



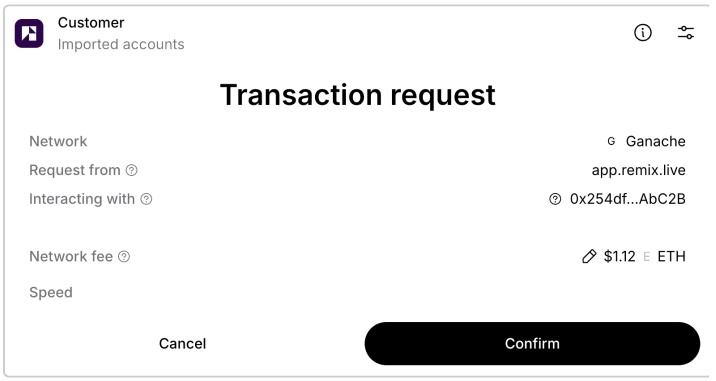
Stage 1: `addBatch(1, "Alphonso Mango", 500)` on `FarmerContract`, approved by the Farmer account.



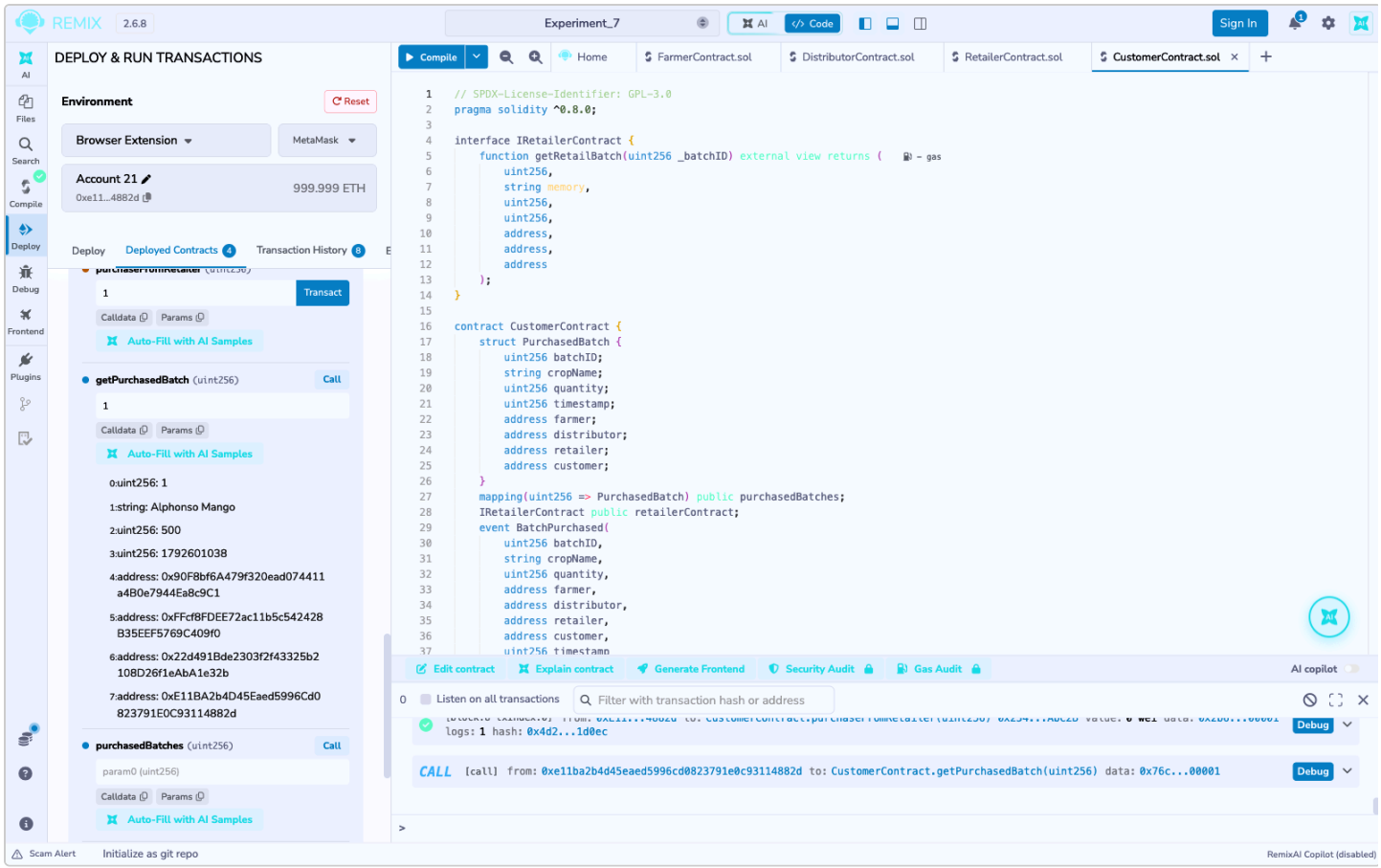
Stage 2: `receiveBatch(1)` on `DistributorContract`, approved by the Distributor.



Stage 3: `receiveFromDistributor(1)` on `RetailerContract`, approved by the Retailer.



Stage 4: `purchaseFromRetailer(1)` on `CustomerContract`, approved by the Customer.



`getPurchasedBatch(1)` *returns batch 1, "Alphonso Mango", 500 kg, with four different addresses recorded: the farmer (0x90F8…), distributor (0xFFcf…), retailer (0x22d4…) and customer (0xE11B…).*

Transaction details on Ganache (Task 6)

```
eth_getBalance
eth_getBalance
eth_getBalance
eth_getBalance
eth_blockNumber
eth_getCode
eth_gasPrice
eth_estimateGas
eth_blockNumber
eth_getCode
eth_call
eth_call
eth_call
eth_call
eth_call
eth_call
eth_call
eth_call
eth_call
eth_call
eth_blockNumber
eth_call
eth_call
eth_call
eth_call
eth_call
eth_getTransactionCount
eth_sendRawTransaction

  Transaction: 0xc34e5fc6acd9a645835fb1d1df0daf575a90a5a7513f05ebc4d733a8a2d3a0b0
  Gas usage: 229346
  Block number: 8
  Block time: Wed Oct 21 2026 22:13:58 GMT+0530 (India Standard Time)

eth_getTransactionReceipt
eth_blockNumber
eth_getBlockByNumber
eth_getBalance
eth_getTransactionReceipt
eth_getBlockByHash
eth_getTransactionByHash
eth_getBalance
eth_getBalance
eth_getBalance
eth_getBalance
eth_getBalance
eth_getBalance
eth_getBalance
eth_getCode
eth_gasPrice
eth_call
eth_blockNumber
eth_blockNumber
eth_blockNumber
```

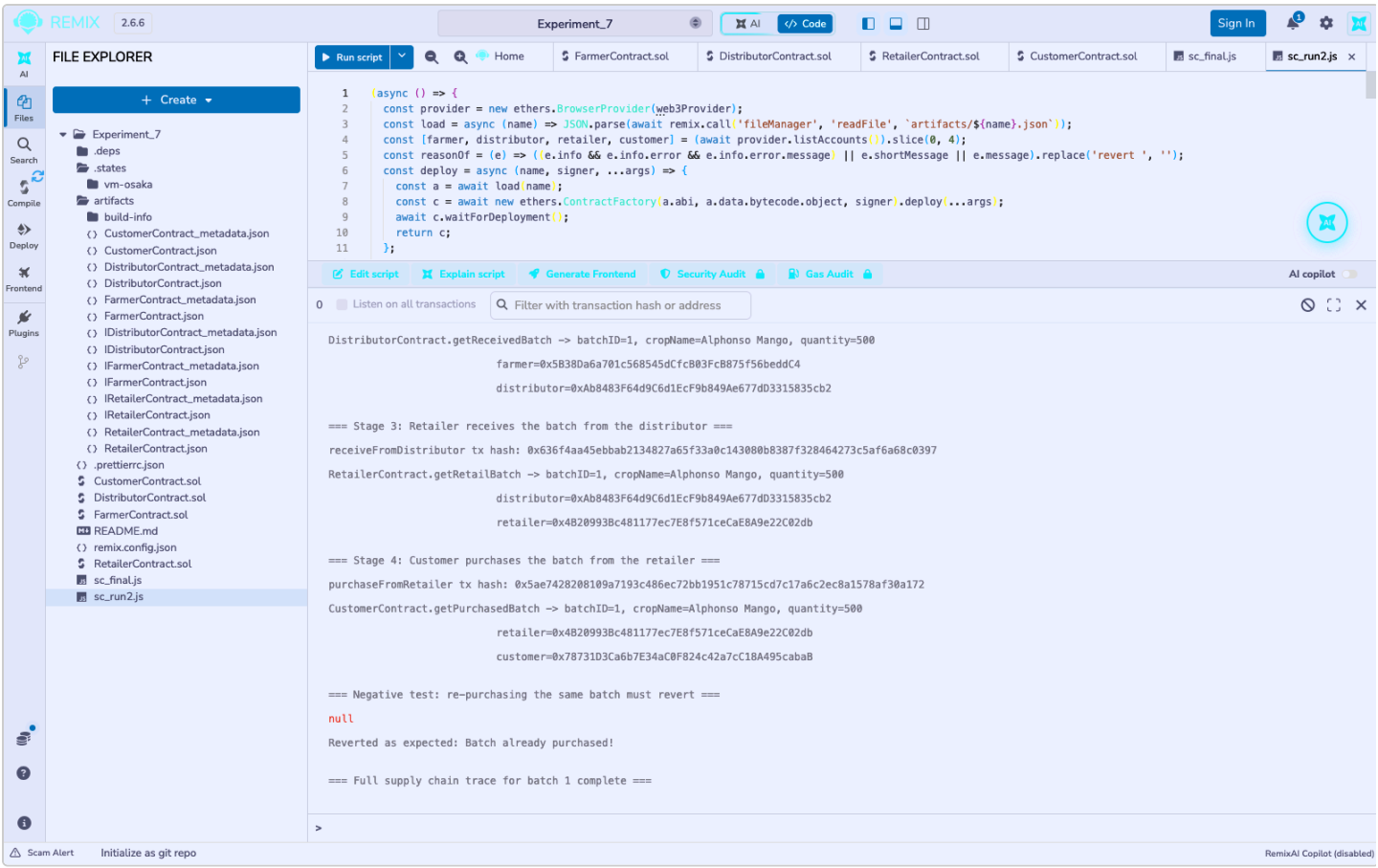
Ganache logs each mined transaction with its hash, gas used, block number and block time (here block 8, the customer's purchase, 229,346 gas).

```
Experiment_7_Output $ node scripts/ganache_tx_list.js
Ganache  chainId=5777  latest block=8

BLOCK  TX HASH        FROM         TO                   GAS USED  STATUS
1      0xb3ef7242c5...  Farmer       new 0xe78A0F...      734422    success
2      0xf38248ef81...  Farmer       new 0x5b1869...      945475    success
3      0x28ce263134...  Farmer       new 0xCfEB86...      1015064   success
4      0x74a374413c...  Farmer       new 0x254dff...      1069108   success
5      0xdc5fb3093d...  Farmer       0xe78A0F7E...        138286    success
6      0x6305bbbb67...  Distributor  0x5b1869D9...        179002    success
7      0x14d2b0ab84...  Retailer     0xCfEB869F...        204156    success
8      0xc34e5fc6ac...  Customer     0x254dffcd...        229346    success

BALANCES
Farmer       0x90F8bf6A479f320ead074411a4B0e7944Ea8c9C1  999.99219529 ETH
Distributor  0xFFcf8FDEE72ac11b5c542428B35EEF5769C409f0  999.999641996 ETH
Retailer     0x22d491Bde2303f2f43325b2108D26f1eAbA1e32b  999.999591688 ETH
Customer     0xE11BA2b4D45Eaed5996Cd0823791E0C93114882d  999.999541308 ETH
Experiment_7_Output $
```

A script reads the chain over RPC and lists all eight blocks: four contract creations by the Farmer account, then `addBatch`, `receiveBatch`, `receiveFromDistributor` and `purchaseFromRetailer`, each with gas used and a success status, followed by the account balances.



Negative test on the Remix VM: a second purchase of the same batch reverts with “Batch already purchased!”.

Conclusion

This experiment started a local Ethereum blockchain with Ganache, connected four of its accounts to MetaMask as the Farmer, Distributor, Retailer and Customer, and used Remix IDE’s Browser Extension environment to deploy a 4-contract supply-chain traceability system through MetaMask. Each deployment and each stage transition was an approval screen in MetaMask and a separate mined transaction on the Ganache chain, signed by the account of the role that performed it. The final record for batch 1 carries the four different role addresses, which shows that each stage verified the previous stage’s on-chain data through its interface before recording its own step. The transactions, gas usage and balances were checked on the Ganache chain, and the negative test confirmed that a batch cannot be purchased twice.